

Quantum ISD (updated ver.)

<https://youtu.be/AC6wR9mCrEw>

Quantum Information Set Decoding (QISD)

- 코드 기반 암호에 대한 공격 기법인 ISD에 Grover search 적용
- 이전 과제에서 해왔던 Prange ISD 사용하고 내부 모듈 약간 변경
 - Information set에 대한 역행렬을 구하는 연산을 신드롬 C에도 적용하여 나오는 결과 벡터의 해밍웨이트가 조건을 만족한다면 공격에 성공
- ISD 오라클은 다음과 같이 크게 3가지 단계로 구성
 - Information set 구축
 - Information set에 대한 역행렬 계산 (가우스 소거) → 가장 연산량이 많음
 - 해밍 웨이트 계산

Information set 구축

- 패리티체크 행렬 H 에 대한 부분 집합인 H_{n-k} 를 생성
 - 실제 파라미터에서 대규모 행렬 (e.g., $n=17669$, $k=128$ (HQC-1))
 - 여기서는 toy example로 실험 ($n=16$, $k=8$)

오른쪽 그림에서 $n-k$ 개 의미

- 전체 n 열 중 H 의 부분집합으로 k 개의 열을 선택
 - 전체 H 에 대한 공격이 아님 (수학적으로 그래도 된다고 함)
 - 가능한 경우의 information set을 만들기 위해
 n 개 중 k 개의 열이 뽑힐 경우를 중첩 상태로 둠
 $\rightarrow$ dicke state 사용
 - 선택된 열들은 따로 추출되어 이후 역행렬 계산에 사용 (가우스 소거)
- 선택된 인덱스에 따라 H_{n-k} 추출

1로 세팅된 열이 선택된 열을 의미
여기서는 7번째~14번째 열이 선택

$$|D_8^{16}\rangle: \quad 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0$$

$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow$

$$H = \left(\begin{array}{cccccc|cccccc} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \end{array} \right)$$

Information set

Information set 구축

K행이 있긴 하지만 그 중
첫번째 행에 선택된 열 정보 존재

Information set에
첫번째로 선택된 열은
H의 7번째 열임을 의미

Selected k-columns (Superposition/Selection)

0 0 0 0 0 0 1 1 1 1 1 1 1 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Permutation matrix (perm as k*n)

0 0 0 0 0 0 1 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 1 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 1 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 1 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 1 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 1 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 1 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 1 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 1

- 구현상 이 단계에서 2개의 모듈이 필요

- 인덱스 식별 (H에서 어떤 열이 선택된 것인지 알기 위함)
 - 0000001111111100 으로 두면 선택된 열에 대한 정보를 알기 어려움
- 해당 열 추출 (information set 구성)

- 인덱스 식별

- 우측 그림과 같이 선택된 열을 표현하는 n 큐비트를 k행 n열의 형태로 변경 → k행: 선택된 개수, n열: 선택 정보 표시
 - 0행에 미선택 (0) / 선택 (1) 정보
 - 1~7행은 one-hot 인코딩 형태로 출력하기 위해 할당해둔 클린 큐비트 (모두 0)
 - 따라서, 하나의 열에 1이 최대 한 개만 존재 (0행에만 정보가 있음)
- 인덱스를 식별하려면 한 행에 하나의 선택 정보만 있어야 함 → 0행의 1들을 아래 행으로 끌고 내려와야하며, 인접 행간의 swap을 수행
 - 현재 열에 대한 컨트롤 큐비트에 따라 우측 열을 swap
- 이전 방식은 하나의 열에서 모든 행을 다 컨트롤 큐비트 삼아서 우측을 인접 swap → 동작은 맞지만 swap 연산이 중복되는 부분이 생김
- 따라서, swap 연산 전에 하나의 열에 대해 1이 있는지를 체크하여 저장하고, 해당 결과에 따라 swap
 - 1 존재 여부는 단순 XOR (하위 행들이 모두 0이므로 가능), 병렬 가능
 - 하나의 열에 대한 모든 행을 컨트롤로 사용하며 연산하지 않아도 됨 → 중복된 스왑 제거 + 컨트롤 큐비트 복사를 통해 열 단위 병렬화 적용 (안실라는 다음 단계에서 빌려옴)

Information set 구축

- $k \times k$ 크기의 information set 생성
 - 인덱스에 따라 H에서 선택된 열들을 복사해야함
 - 따라서, 인덱스 및 H 행렬에 대한 AND 연산이 필요 → 결과가 information set
 - 인덱스 행렬 $[p][i]$ & $H[j][i] \rightarrow Hk[j][p]$
몇 번째 선택? H의 행 선택됐던 순서대로 복사되므로
- 예를 들어, 인덱스 행렬[2]이 다음과 같다면 세번째로 선택됐던 열이 H의 7번째 열이라는 의미이고, H의 7번째 열에 대한 모든 행을 Hk의 세번째 열의 모든 행에 복사

```
0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
```

- 전체 구조는 그대로 두고, 인덱스만 변경하여 최대한 겹치지 않는 큐비트들에 한번에 많이 접근하도록 해서 병렬성 높임
 - 기존 구현에서 3중 반복문으로 간단히 구현되어 있음
→ Hk가 사용하지 않는 i에 대한 반복이 가장 내부 반복이어서 같은 큐비트에 계속 접근하는 형태일 수도 있을 것 같았음
→ 컴파일러가 어느정도 병렬화 시켜주는 하겠지만 약간 변경함
- 동일 i에 대해서 최대한 겹치지 않는 큐비트를 동시에 사용하도록 해야함
 - 두 컨트롤 큐비트 (인덱스 행렬과 H)과 겹치지 않는 j,p를 선택하도록 함 → k개의 그룹으로 나눌 수 있음
 - 예시: $\text{index}(j,p) \Rightarrow (0,0), (1,1), (2,2), (3,3) \ // \ (0,1), (1,2), (2,3), (3,4) \ // \ (0,2), (1,3), (2,0), (3,1) \ // \ (0,3), (1,0), (2,1), (3,2)$

Gauss-Jordan Elimination

- 해당 부분은 이미 병렬화가 잘 되어 있어서 변경하지 않음

해밍 웨이트 확인

- 앞에서 얻은 Information set의 역행렬을 구하는 연산을 신드롬 C에도 적용한 결과 벡터에 대한 해밍 웨이트 체크
- 기존 구현은 트리방식으로 덧셈하여 모든 1을 카운트함
 - 8비트에 대한 해밍 웨이트 계산을 예로 들면, 4비트씩 나누어 덧셈 → 4비트 내부에서는 2비트로 나누어 계산하는 트리 방식
- 해당 방식은 k가 커질수록 덧셈 호출 횟수도 증가함
- 따라서, QCSA (멀티 오퍼랜드 덧셈)을 적용하여 한 번에 모든 비트를 더함
 - k비트에 대한 해밍웨이트는 k개의 각 비트를 다 더하면 됨
 - 0 0 1 1 0 0 0 0 은 $0+0+1+1+0+0+0+0$ 이 되는 것
 - 따라서 1비트 k개를 한 번에 더하면 해밍웨이트를 구할 수 있음
- 필요한 안실라 큐비트들은 이전 단계인 가우스 소거에서 사용한 충분히 많은 수의 안실라를 빌려올 수 있음

결과

- 큐비트 수 동일, 게이트 수 약간 감소, Full depth 약 17퍼 감소

```
Syndrome weight  
0x2
```

```
Gate counts:
```

```
  Allocate : 543  
  CSwap : 840  
  CX : 10954  
  Deallocate : 543  
  H : 756  
  Measure : 86  
  S : 20  
  T : 4552  
  T^\dagger : 6056  
  X : 100
```

```
Max. width (number of qubits) : 506.
```

```
Depth : 881
```

```
Error Locations(Syndrome)
```

```
0 0 1 1 0 0 0 0
```

```
Syndrome weight  
0x2
```

```
Gate counts:
```

```
  Allocate : 543  
  CSwap : 4172  
  CX : 10519  
  Deallocate : 543  
  H : 736  
  Measure : 72  
  T : 4629  
  T^\dagger : 6133  
  X : 80
```

```
Max. width (number of qubits) : 467.
```

```
Depth : 1061
```

```
Error Locations(Syndrome)
```

```
0 0 1 1 0 0 0 0
```


계획

- 일단 토이 예제에서는 약간의 성능 향상 있음
- 더 큰 파라미터에서의 개선은 어느정도인지 확인할 예정
- 사실 QISD의 이론에 대해 아주 잘 아는 것은 아니어서 놓친 부분이 있는지 확인해볼 예정

Thank you.

감사합니다.